



DOCUMENTATION TECHNIQUE

Architecture & État du Prototype

Référence technique de la maquette Kamili EduCampus : architecture logicielle, système de design, cartographie fonctionnelle des 8 espaces utilisateurs et feuille de route vers la production.

React 19

Vite 8

React Router 7

86 écrans

27 composants

37 modules de données

Ce que ce document couvre

Ce document décrit l'état technique actuel de Kamili EduCampus : une maquette (prototype) applicative interactive, développée en React, qui modélise l'intégralité des parcours utilisateurs du produit final à l'aide de données de démonstration. Il s'adresse aux équipes techniques, aux partenaires d'intégration et aux évaluateurs souhaitant comprendre l'architecture existante et le chemin restant vers une mise en production.

8

ESPACES
UTILISATEURS

86

ÉCRANS / PAGES

27

COMPOSANTS UI

37

MODULES DE
DONNÉES

≈23k

LIGNES DE CODE

Statut du projet

Important : il s'agit à ce stade d'une maquette front-end. Toutes les données (élèves, enseignants, établissements, notes, paiements...) sont simulées et stockées localement dans le code source (dossier `src/data`), à des fins de démonstration. Aucun back-end, aucune base de données réelle et aucune intégration de paiement ne sont connectés à ce jour. La section « Feuille de route vers la production » détaille les travaux restants.

Ce que la maquette démontre déjà

- **Couverture fonctionnelle complète :** navigation, tableaux de bord, graphiques et parcours pour les 8 profils d'utilisateurs du produit final.
- **Système de design abouti :** charte graphique bordeaux & or, thèmes clair/sombre, composants réutilisables cohérents.
- **Architecture modulaire :** séparation claire entre pages, composants, données et mise en page, facilitant le branchement d'un vrai back-end.
- **Modèle de données réaliste :** jeux de données calibrés sur les ordres de grandeur du système éducatif camerounais (élèves, établissements, enseignants, résultats par région).

Sauf mention contraire, les statistiques de ce document (nombre d'écrans, de composants, de lignes de code) ont été mesurées directement sur le code source du dépôt à la date de publication.

Stack technique & principes

Stack technologique

COUCHE	TECHNOLOGIE	RÔLE
Framework UI	React 19	Composition de l'interface en composants réutilisables
Outil de build	Vite 8	Serveur de développement rapide (HMR) et build de production optimisé
Routage	React Router 7	Navigation déclarative multi-espaces (createBrowserRouter)
Qualité de code	Oxlint	Linting rapide (Rust) intégré au flux de développement
Style	CSS natif + design tokens	Variables CSS pour couleurs, espacements, typographie ; pas de dépendance UI externe
Module system	ES Modules (type: module)	Imports natifs, compatibles avec l'écosystème moderne

Principes d'architecture

- **Composants d'abord** : une bibliothèque de 27 composants d'interface génériques (boutons, tableaux, cartes, graphiques...) partagés par les 8 espaces applicatifs.
- **Données découplées de l'affichage** : chaque module de src/data expose des données et fonctions d'accès pures, prêtes à être remplacées par des appels API sans toucher aux pages.
- **Layouts par profil** : chaque espace (Parent, Élève, Enseignant...) possède son propre binôme Sidebar/Topbar, injecté dans un DashboardLayout commun.
- **Design tokens centralisés** : couleurs, rayons, ombres, espacements et typographie définis une seule fois (src/styles/tokens) et consommés partout via des variables CSS.

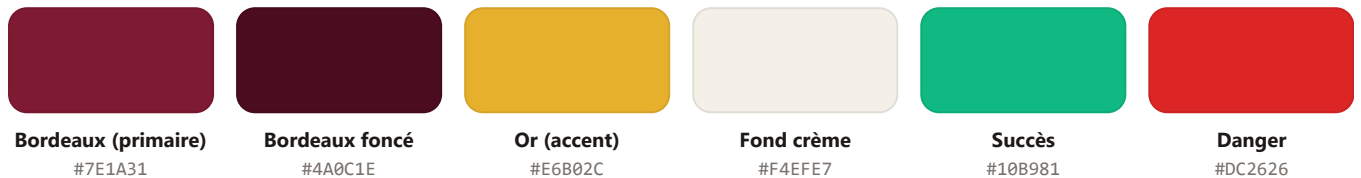
Structure des dossiers (extrait)

```
src/
├─ components/      # 27 composants UI réutilisables
├─ layouts/         # AuthLayout, DashboardLayout, EmptyLayout
├─ pages/
│   ├─ Home, Establishments, auth/
│   └─ dashboards/
│       ├─ parent/ student/ teacher/ direction/
│       └─ ministry/ minesup/ teacher-admin/ visitor/
├─ data/            # 37 modules de données de démonstration
├─ routes/          # index.jsx – arborescence des routes
├─ styles/
│   └─ tokens/      # colors, spacing, radius, shadows, typography, breakpoints
├─ hooks/, contexts/, utils/, features/
└─ App.jsx, main.jsx
```

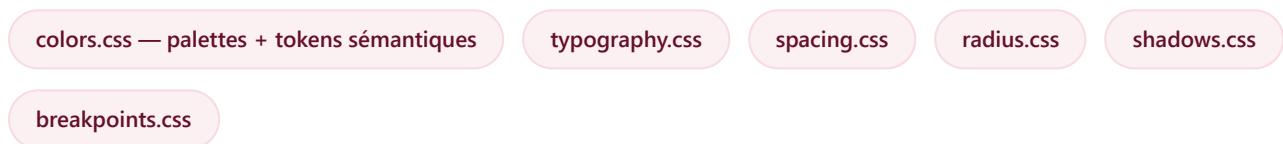
Une identité visuelle cohérente

L'ensemble de l'interface repose sur des tokens de design centralisés (fichiers CSS dans `src/styles/tokens`), garantissant une cohérence visuelle sur les 8 espaces et une bascule automatique entre thème clair et thème sombre.

Palette de marque



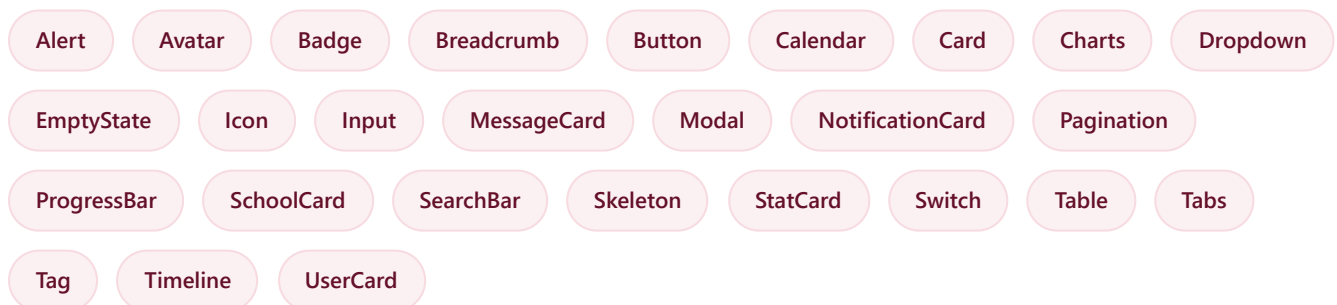
Tokens disponibles



Thèmes clair / sombre

Les couleurs sémantiques (fond, texte, bordures, états) sont redéfinies pour le thème sombre via `@media` (`prefers-color-scheme: dark`) et un attribut `data-theme` permettant une bascule manuelle, sans dupliquer la logique des composants.

Bibliothèque de composants (27)



8 espaces applicatifs

Chaque profil dispose de son propre sous-arbre de routes (src/routes/index.jsx), avec sidebar, topbar et pages dédiées.

Parent

13 pages

Suivi multi-enfants

Accueil, aperçu, statistiques, enfants, paiements, présence, notes, résultats, agenda, messagerie, notifications, profil, inscription.

Élève

13 pages

Vie scolaire de l'élève

Accueil, notes, emploi du temps, devoirs, bibliothèque, hébergement, bourses, paiements, portefeuille, messagerie, agenda, notifications, profil.

Enseignant

10 pages

Gestion pédagogique

Classes, présence, notes, cahier de texte, planning, parents, messagerie, examens, statistiques, profil.

Direction

13 pages

Pilotage d'établissement

Accueil, inscriptions, enseignants, classes, présence, résultats, paiements, finances, graphiques, alertes, rapports, profil, permissions.

MINEDUB / MINESEC

6 pages

Supervision ministérielle

Supervision, établissements, enseignants, résultats, graphiques, alertes.

MINESUP

6 pages

Enseignement supérieur

Accueil, universités, enseignants, résultats, graphiques, alertes.

Admin pédagogique

1 espace

Pilotage transverse

Tableau de bord dédié au pilotage des établissements partenaires.

Visiteur / Public

3 pages

Découverte & accès

Accueil, annuaire des établissements, tableau de bord visiteur, parcours d'authentification (connexion, OTP, mot de passe oublié, sélection de rôle).

Modules de données (extrait, 37 au total)

national.js — annuaire des établissements

ministry.js — KPIs régionaux & ministériels

users.js / roles.js

payments.js / wallet.js

grades.js / results.js

attendance.js

scholarships.js / housing.js / library.js

timetable.js / homework.js / agenda.js

messages.js / notifications.js

... + variantes par profil (student*, teacher*, parent*)

De la maquette à la plateforme de production

Le prototype couvre l'intégralité des parcours utilisateurs ; la mise en production nécessite de connecter cette interface à une infrastructure back-end réelle. Chantiers prioritaires, financés par la Phase 1 du plan de déploiement (voir dossier budgétaire) :

1. Back-end & API

Conception d'une API (REST ou GraphQL) sécurisée, remplaçant progressivement les modules de src/data par de vrais appels réseau, sans changement de contrat pour les pages existantes.

2. Base de données

Modélisation d'une base relationnelle (élèves, établissements, enseignants, paiements, résultats) reprenant la structure déjà éprouvée par les modules de données de la maquette.

3. Authentification & gestion des rôles

Remplacement du parcours de connexion simulé par une authentification réelle (OTP par SMS, gestion de session), en s'appuyant sur le modèle de rôles et de permissions déjà présent dans la maquette (espace Direction > Permissions).

4. Paiements Mobile Money

Intégration des API Orange Money et MTN MoMo pour la collecte de la micro-cotisation et des frais scolaires, avec réconciliation automatisée et reçus numériques.

5. Hébergement souverain & sécurité

Déploiement sur une infrastructure cloud conforme à la réglementation camerounaise, chiffrement des données sensibles, sauvegardes régulières, audits de sécurité.

6. Observabilité, tests & CI/CD

Mise en place de tests automatisés, d'un pipeline d'intégration/déploiement continu et d'outils de supervision applicative avant l'ouverture du pilote aux 50 premiers établissements.

7. Scalabilité multi-établissements

Architecture multi-tenant permettant l'onboarding progressif de milliers d'établissements sans dégradation de performance, à l'approche des phases régionale et nationale.

Note de méthode : l'architecture actuelle (composants découplés des données) a été conçue précisément pour minimiser le travail de réécriture de l'interface lors du branchement du back-end — l'essentiel de l'effort de la Phase 1 porte sur le back-end, la sécurité et l'intégration des paiements, pas sur une refonte du front-end.